

Week 4 L4.2
Time Series Classification

COMP3222: Machine Learning Technologies

University of Southampton

COMP3222 Lecture Structure

1. From time series to feature vectors

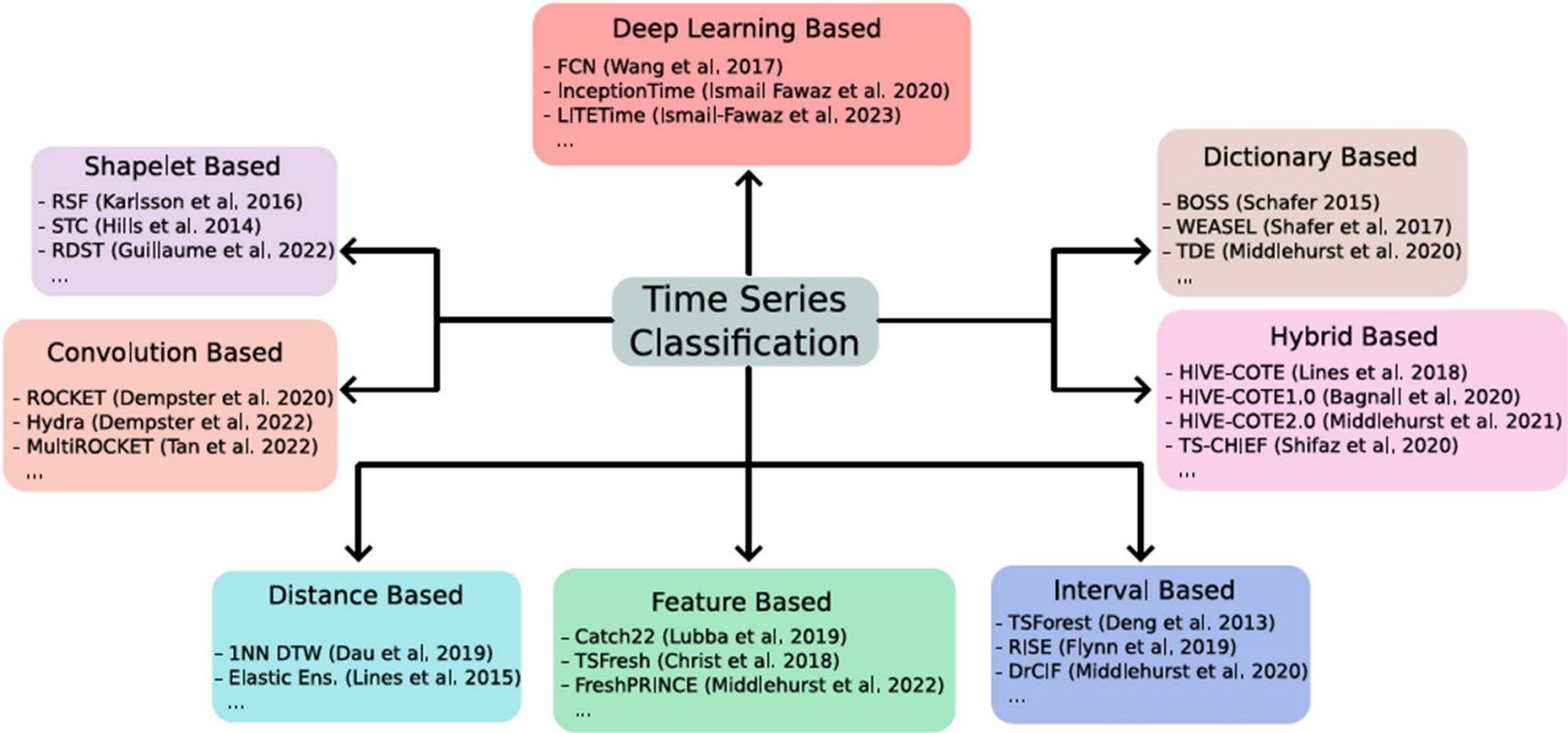
2. Global features and interval forests (**feature-based** and **interval-based**)

3. Features from observed patterns: shapelets and information gain (**shapelet-based**)

4. Features from convolutions/kernels (**convolution-based**)

5. Recurring patterns: (**dictionary-based**)

Taxonomy of Algorithms



Connections to Previous Lectures

Familiar idea	Time series version
Nearest neighbours	Compare series using Euclidean distance or DTW Compare histograms of words using vector space model
Feature engineering	Summarise a whole series or a subsequence
Decision trees and ensembles	Build trees on features from random intervals
Information gain	Select a subsequence that separates the classes

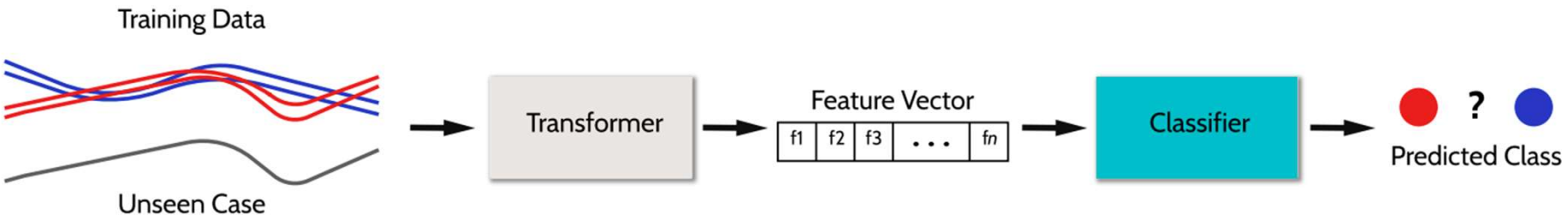
TSC Algorithms: Pipelines

- A time series transform produces a feature vector for each case.

Step 1: Fit the transform on training data, if fitting is required.

Step 2: Transform the training cases and fit a familiar classifier.

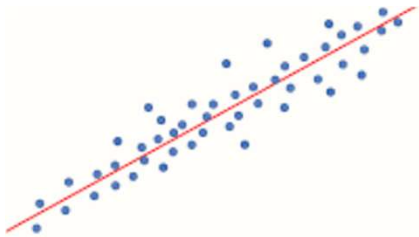
Step 3: Apply the same transform and classifier to new cases.



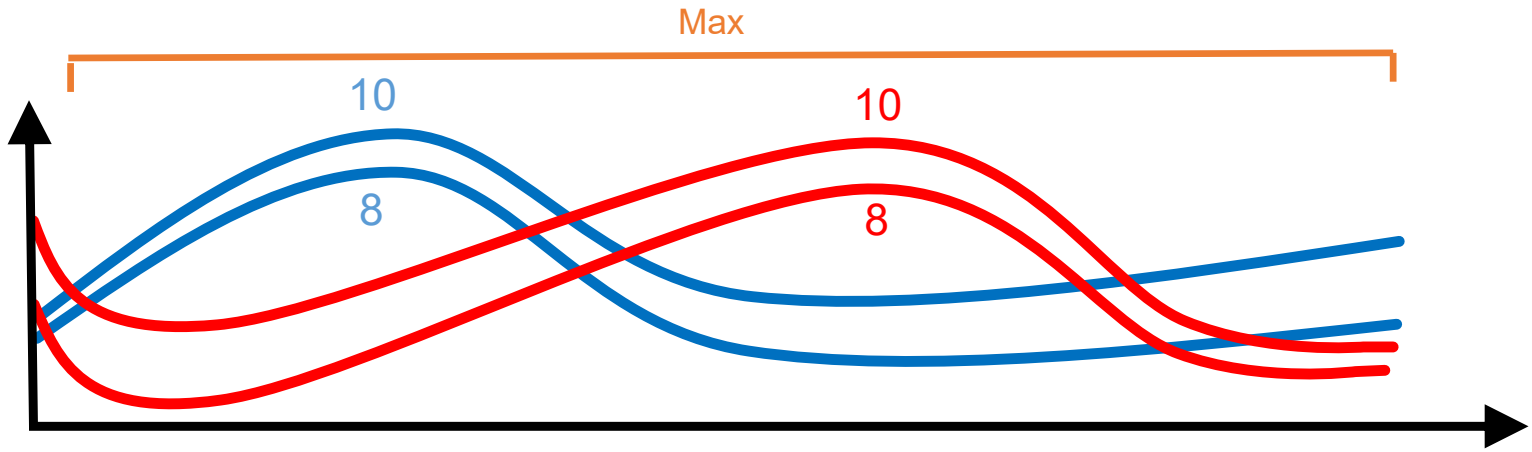
Global Summary Features

Feature	What does it describe?
Mean	Average level
Standard deviation	Variation in values
Minimum and maximum	Extreme values
Slope	Overall increasing or decreasing tendency

Each feature gives one number per channel. Together they form a feature vector.



Time Series Feature Extraction



Max	Min	…	Mean	25% Q	*Class*
10	3	…	5	8	*Blue*
8	2	…	4	6	*Blue*
10	2	…	5	4	*Red*
8	1	…	4	2	*Red*

What Do These Features Lose?

```
a = np.array([0, 0, 2, 2])
b = np.array([0, 2, 0, 2])

# Both have mean = 1 and standard deviation = 1.
# Both have minimum = 0 and maximum = 2.
```

The values are the same, but the ordering is different.

Slope uses temporal information (regression)

Which additional feature could distinguish these series?

Extracting Simple Features

```
def summary_features(X):
    return np.concatenate([
        X.mean(axis=2), X.std(axis=2),
        X.min(axis=2), X.max(axis=2)
    ], axis=1)

F_train = summary_features(X_train)
F_test = summary_features(X_test)
print(F_train.shape)
```

GunPoint has one channel: the result has shape (50, 4).

BasicMotions has six channels: this transform would produce 24 columns

A Classifier on Summary Features

```
raw_rf = RandomForestClassifier()
summary_rf = RandomForestClassifier()
X_train, y_train = load_gunpoint(split=`Train`)
X_test, y_test = load_gunpoint(split=`Test`)
F_train = summary_features(X_train)
F_test = summary_features(X_test)

raw_rf.fit(X_train[:,0,:], y_train)
summary_rf.fit(F_train, y_train)

pred = raw_rf.predict(X_test[:,0,:])
pred = summary_rf.predict(F_test)
```

Same classifier, different representation.

Canonical Time-series Characteristics (Catch22)



Features can describe autocorrelation, changes and repeated patterns

Catch22 provides a compact collection of time series characteristics: Examples

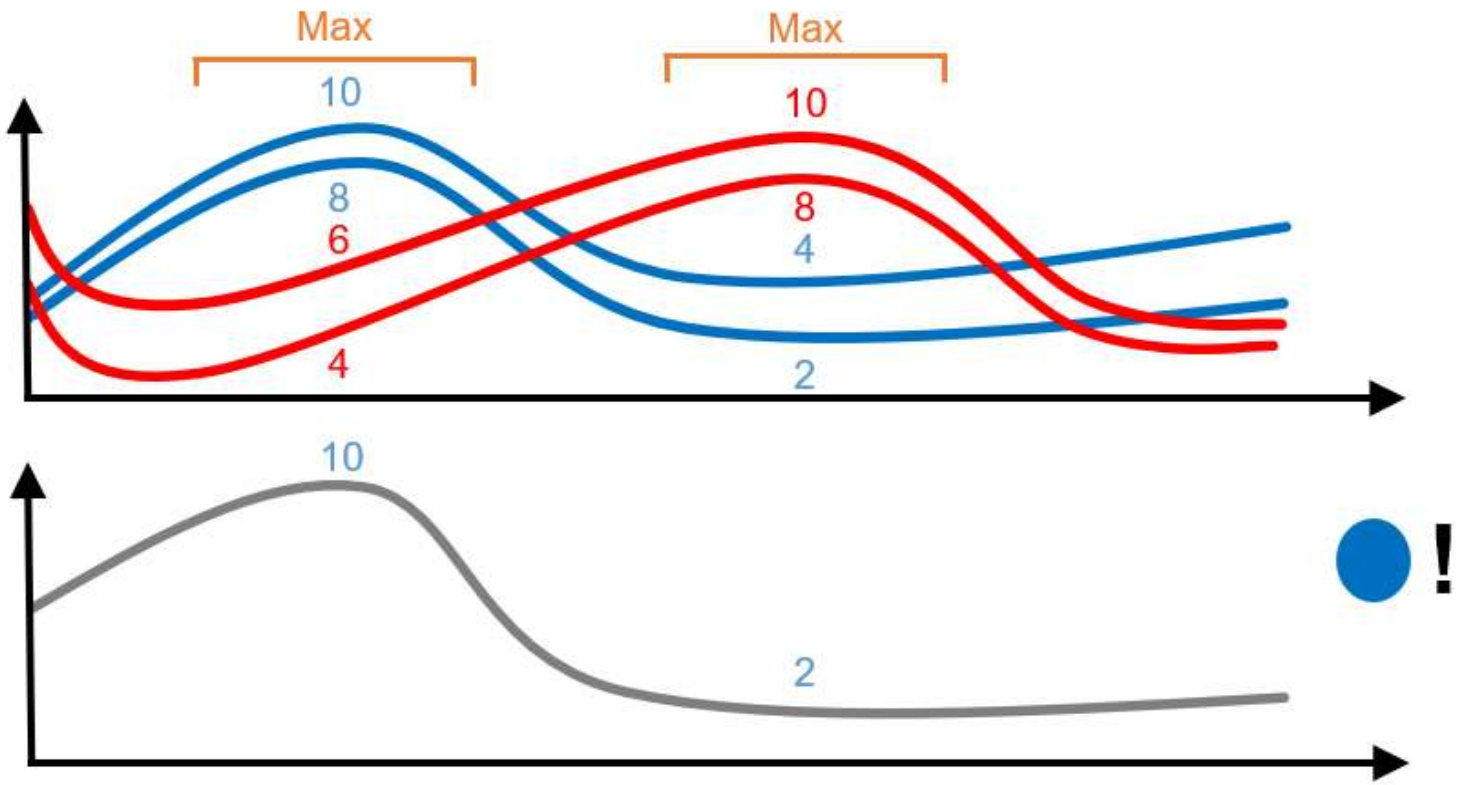
SB_BinaryStats_mean_longstretch1 (stretch_high**)** - Longest period of consecutive values above the mean
SP_Summaries_welch_rect_area_5_1 (low_freq_power**)** - Total power in lowest fifth of frequencies in the Fourier power spectrum

Implemented in *Numba* in *aeon*.

Whole-series features can hide discriminatory features

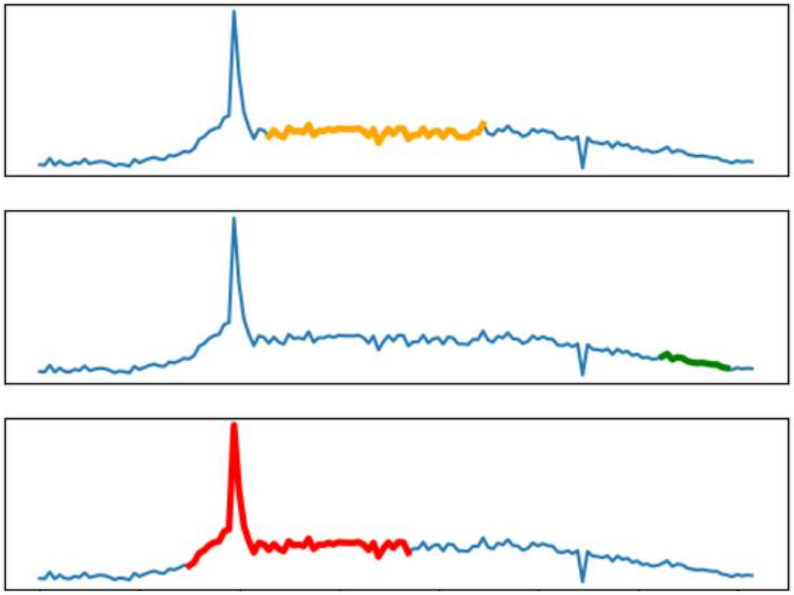
Lubba, C.H., Sethi, S.S., Knaute, P., Schultz, S.R., Fulcher, B.D. and Jones, N.S., 2019. catch22: CAnonical Time-series CHaracteristics: Selected through highly comparative time-series analysis. *Data Mining and Knowledge Discovery*, *33*(6), pp.1821-1852.

Time Series Intervals

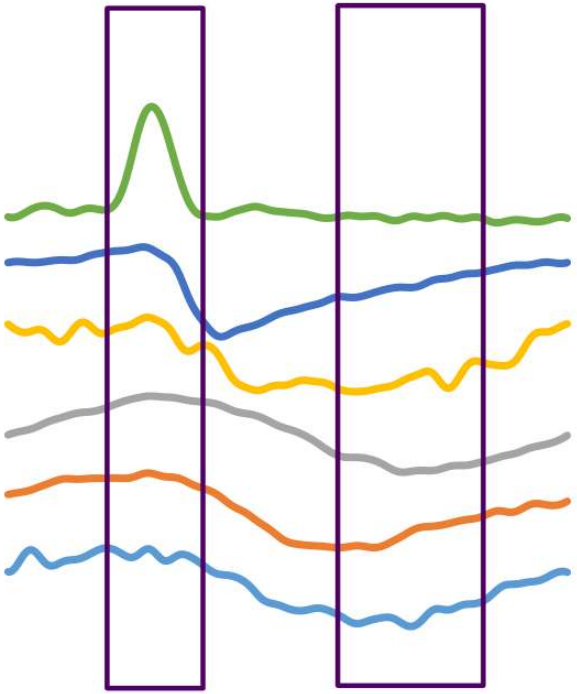


Time Series Intervals

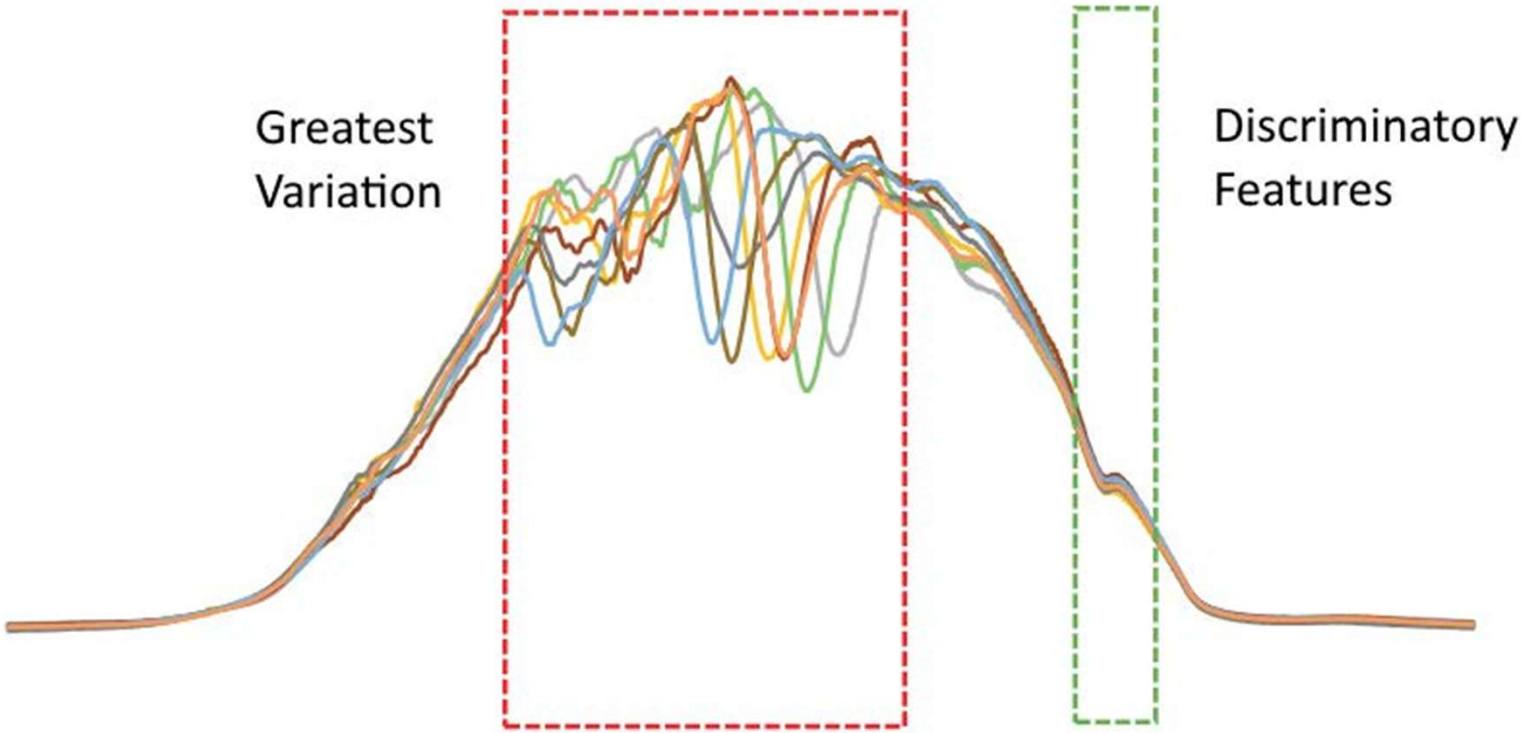
Select multiple time interval subseries



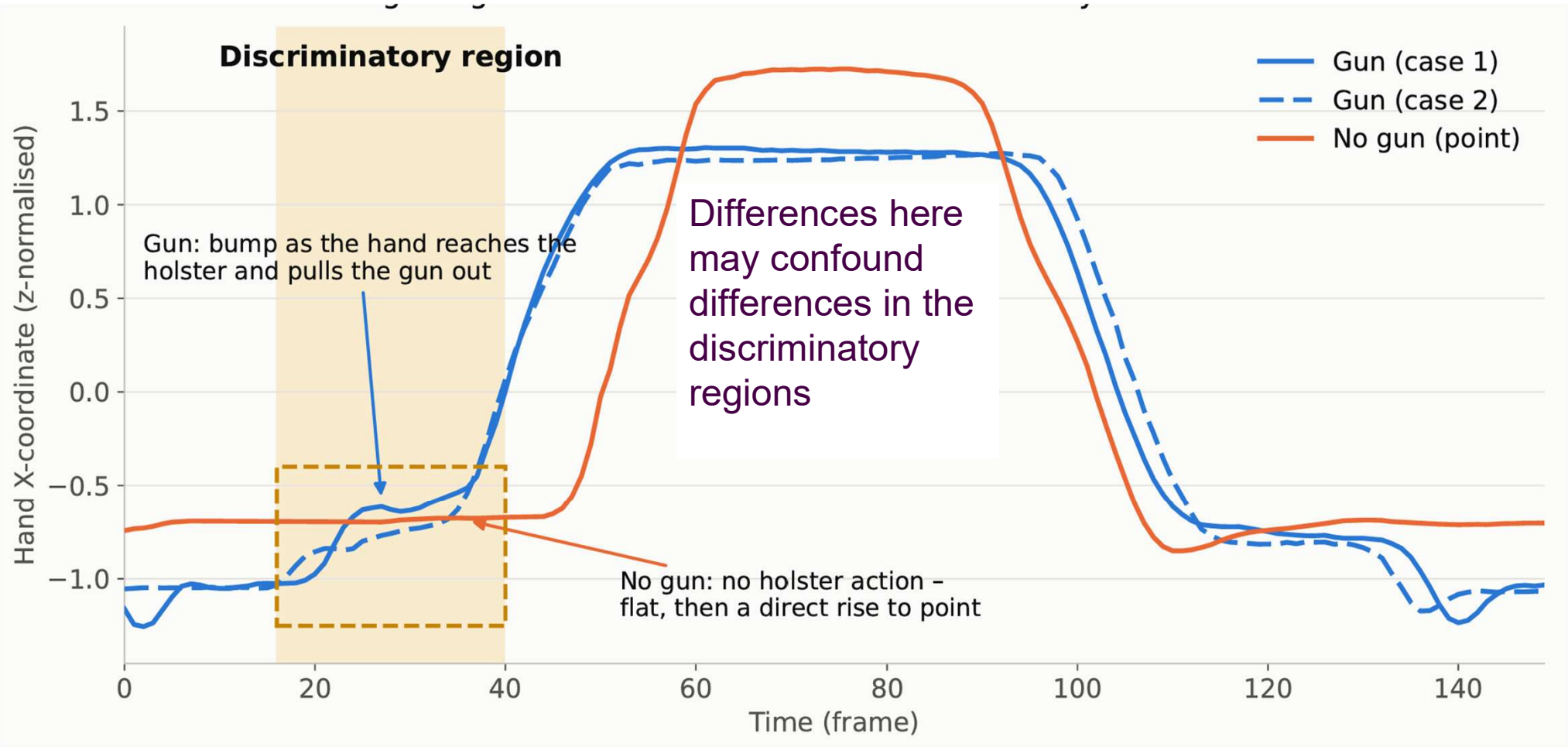
These selected intervals are at the same timepoints for all series



Why Intervals?



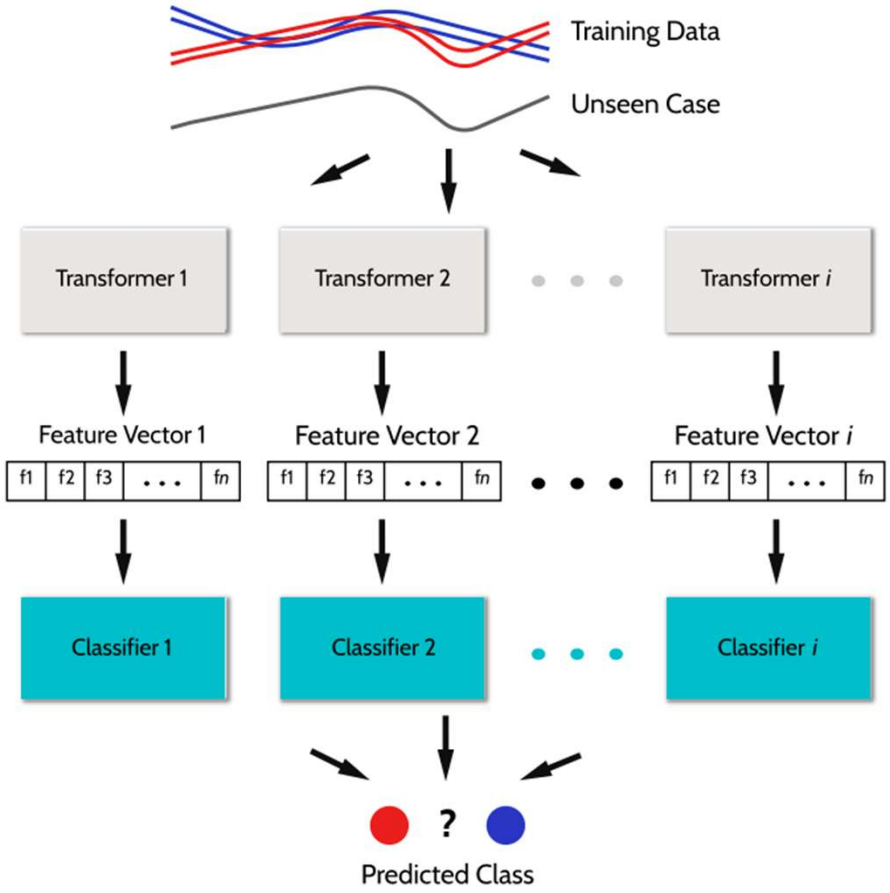
Whole series comparisons may make classification harder



Time Series Forest (TSF)

Key features:

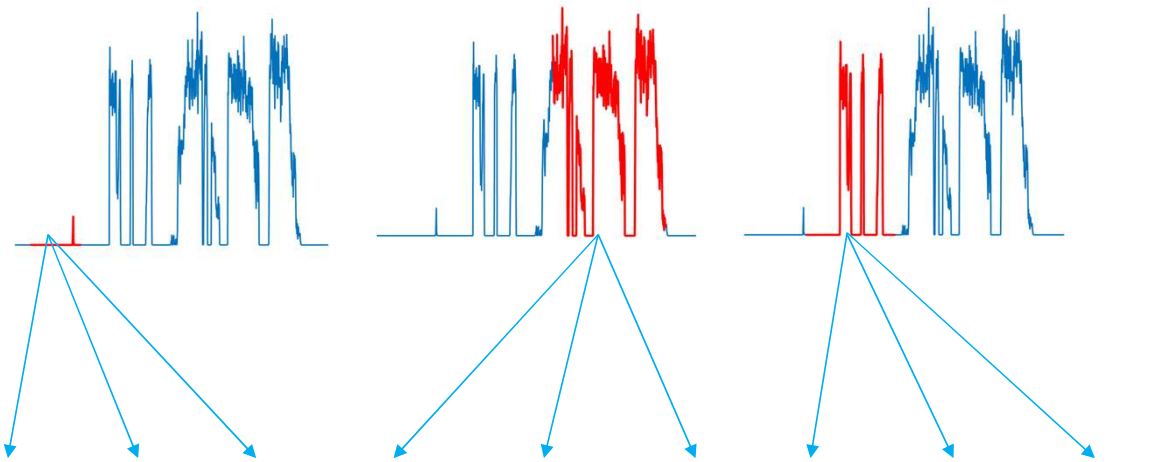
- Ensemble of trees
- Randomly selects intervals
- Extracts 3 simple summary statistics
- Each tree uses a different set of intervals



Time Series Forest (TSF)

For each tree

Randomly select multiple intervals



Time Series

Use interval **mean**, **standard deviation** and **slope** as features

Mean 1	stdev 1	Slope 1	Mean 2	stdev 2	Slope 2	Mean 3	stdev 3	Slope 3

Extracted Features

A Small Interval Example

Series	First interval [0:2]: mean	Second interval [2:4]: mean
A = [0, 0, 2, 2]	0	2
B = [0, 2, 0, 2]	1	1

The whole-series means are identical. The interval means separate the cases.

What happens if the informative pattern moves to a different interval?

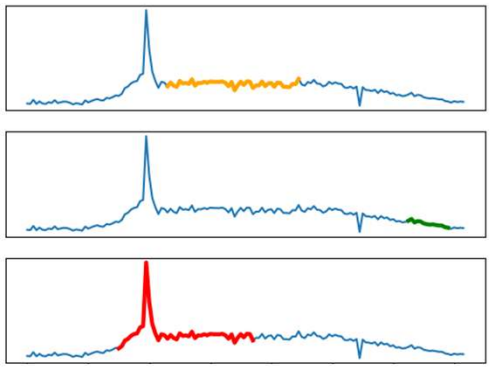
Time Series Forests and Random Forest

- Both combine predictions from an ensemble of trees
- In Time Series Forest, each tree selects random time intervals
- Mean, standard deviation and slope become the tree attributes
- The tree selects useful interval features through its splits
- Diversity comes from choosing different intervals for different trees

Diverse Representation Canonical Interval Forest (DrCIF)

Key features:

- Ensemble of trees and randomly selects intervals like TSF
- Extracts the Catch22 features as well as simple summary statistics
- Extracts interval features from the original series, its periodogram and its first-order differences





Middlehurst, M., Large, J., Flynn, M., Lines, J., Bostrom, A. and Bagnall, A., 2021. HIVE-COTE 2.0: a new meta ensemble for time series classification. *Machine Learning*, *110*(11), pp.3211-3243.
Middlehurst, M., Large, J. and Bagnall, A., 2020, December. The canonical interval forest (CIF) classifier for time series classification. In *2020 IEEE international conference on big data (big data)* (pp. 188-195). IEEE.

Recent Research into Interval Based (information only)

2026

Conferences > 2025 IEEE International Confe...

PULSAR: Advancing Interval-Based Time Series Classification to State-of-the-Art Performance

Publisher: IEEE Cite This PDF

Nestor Cabello ; Lars Kulik All Authors

2025

Journals & Magazines > IEEE Transactions on Industri... > Early Access

Multiview Interval Hypergraph Learning for Time Series Classification and Its Application to Turntable Fault Diagnosis

Publisher: IEEE Cite This PDF

Changchun He ; Xin Huo ; Yuxuan Zhang ; Sebastian Bader All Authors

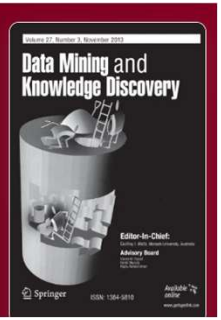
2024

Home > Data Mining and Knowledge Discovery > Article

QUANT: a minimalist interval method for time series classification

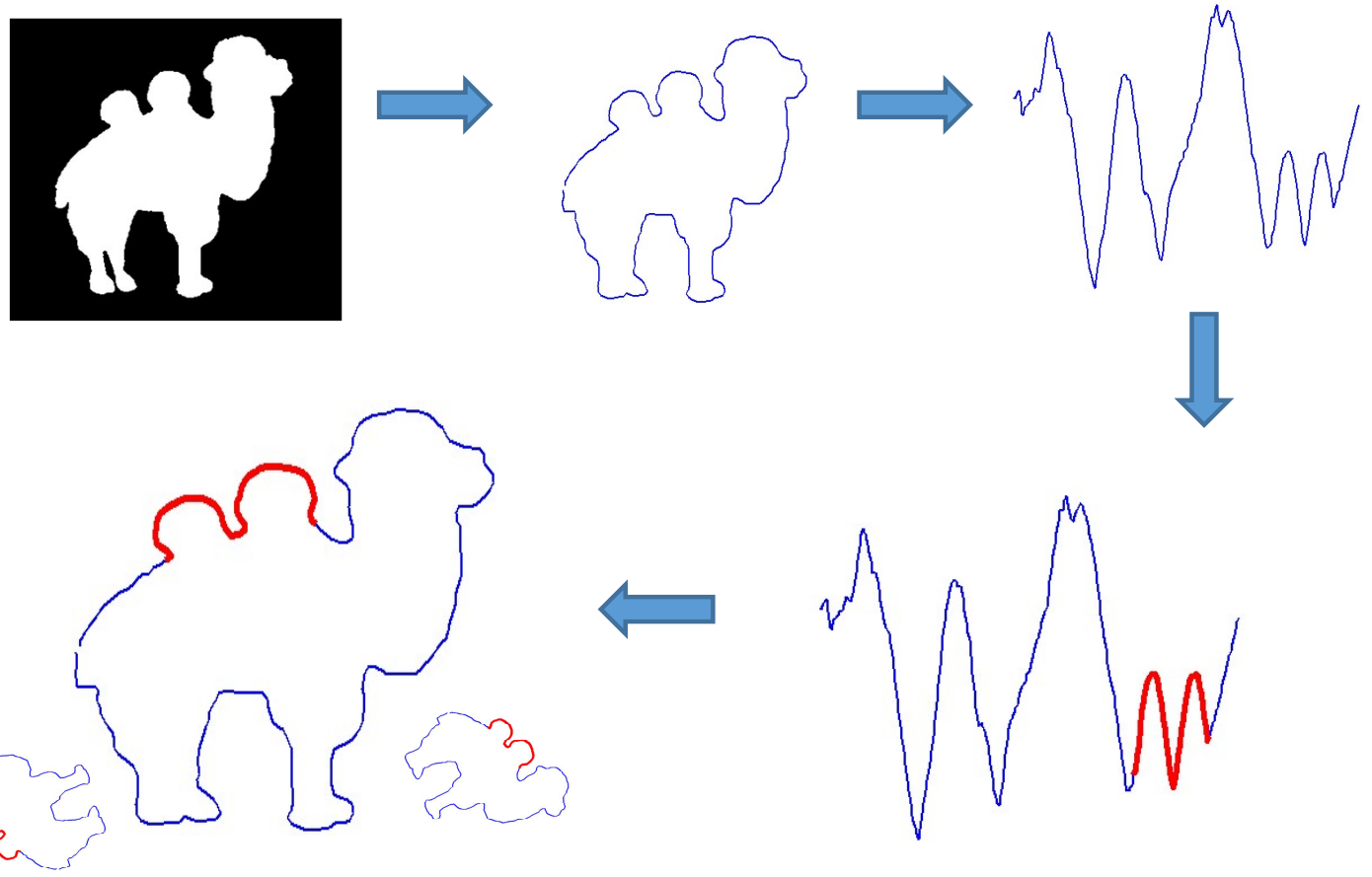
Open access | Published: 22 May 2024

Volume 38, pages 2377–2402 (2024) Cite this article

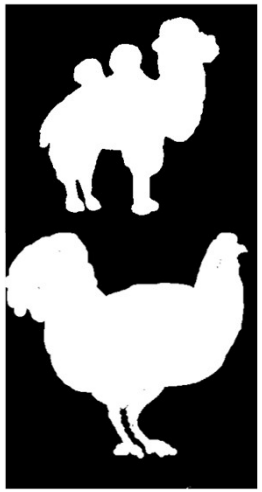


Shapelet-based Algorithms

Shapelets are short patterns that can distinguish between classes



Camel or Chicken?



Shapelets on GunPoint

The same local pattern can occur at different times.

Gun-draw 1



Source pattern

Distance = 0.00

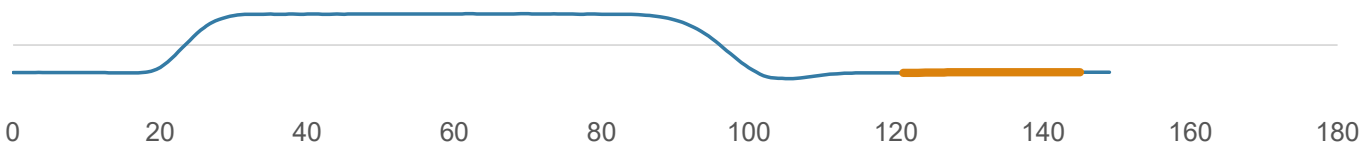
Gun-draw 2



Good match, later

Distance = 1.09

Pointing



Poor match, even at best position

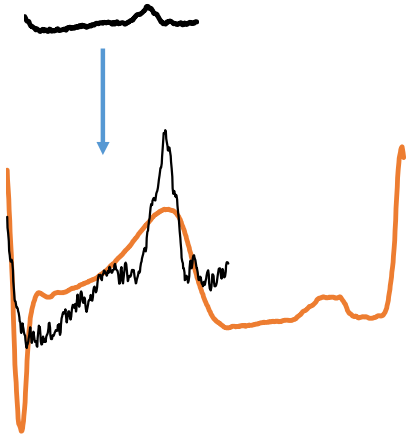
Distance = 2.67

Timepoint (original recording)

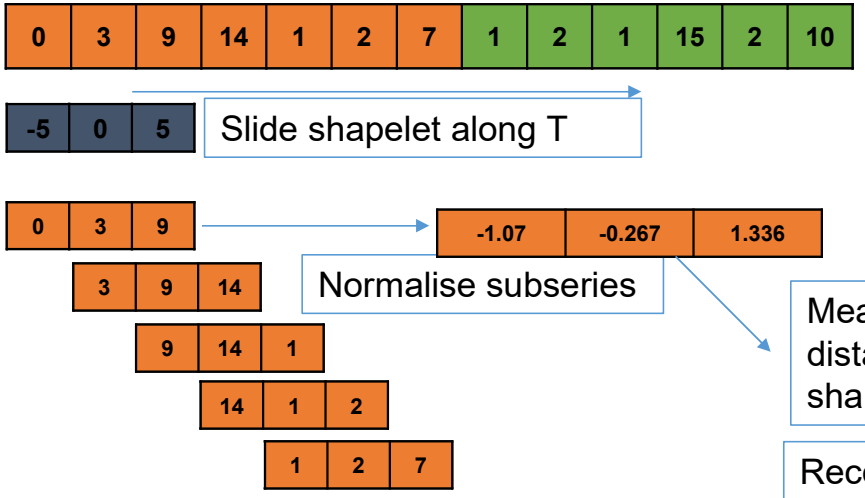
Compare normalised windows at every position. Keep the smallest distance.

Shapelet Distance

We need to measure how likely it is that a shapelet is in any time series



The distance between a subsequence *S* of length *l* and a time series T_i is the minimum distance between *S* and all **normalised** subsequences of T_i of length *l*



$$dist(S,R) = \sum_{i=1}^{l}(s_i - r_i)^2$$

Measure Euclidean distance to normalised shapelet

Record lowest distance so far, then slide along

Algorithm 3 sDist(T, S)

Where T is a time series and S is a shapelet candidate.
$l = |S|$
$min_dist = \infty$
for $p = 0$ **to** $|T| - l + 1$ **do**
  $dist = dist(S, T_p^l)$
  if $dist < min_dist$ **then**
    $min_dist = dist$
return min_dist

Shapelet Quality

For each shapelet we calculate a list of distances from the shapelet to each of the training set time series

The quality of the shapelet is based on how well we can use these distances to predict the class values

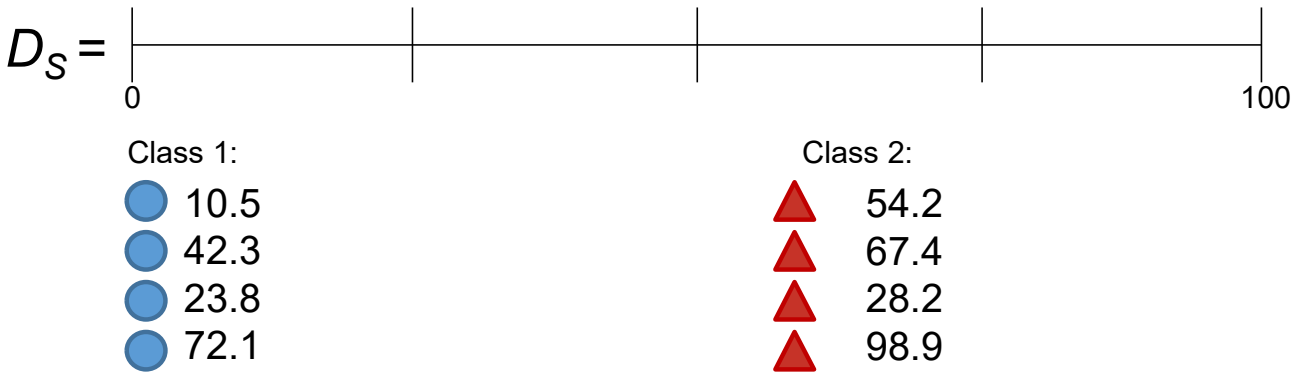
We use binary shapelets that predict a single class (one vs all). Suppose we have eight training cases and get these distances

Class 1:
- 10.5
- 42.3
- 23.8
- 72.1

Class 2:
- 54.2
- 67.4
- 28.2
- 98.9

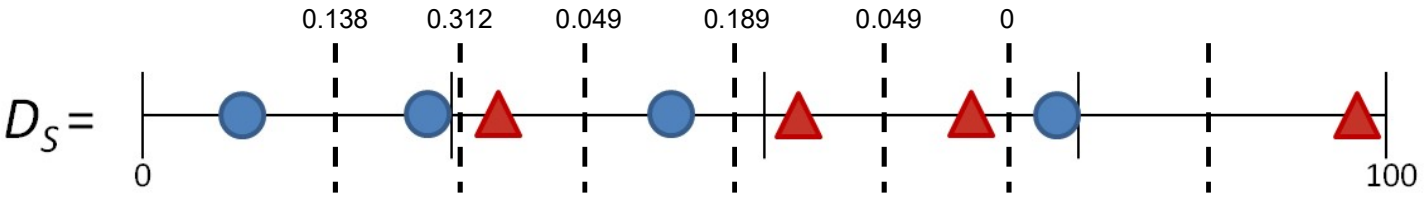
We measure the quality using a measure like information gain

The Order line



Information Gain

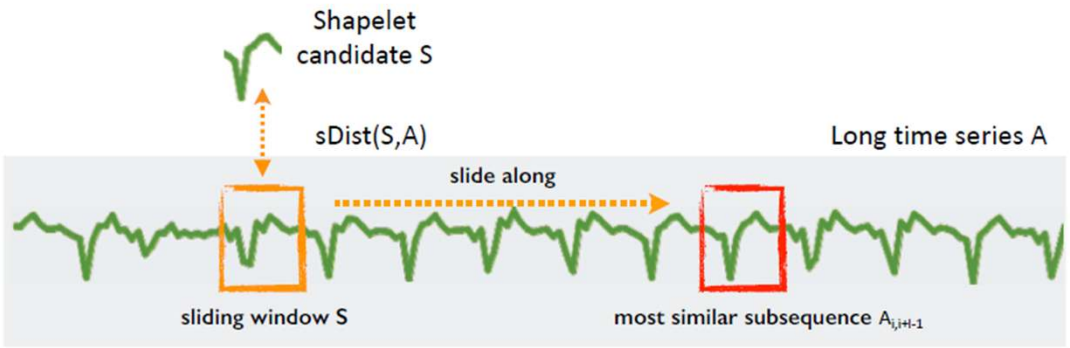
- Now that D_S is sorted, assess all possible split points, where a split point, *sp*, is the average between two consecutive distances.



Shapelet Transform

The shapelet transform converts input time series to a feature vector using the distance to a collection of selected shapelets.

Fit a familiar classifier to the distance features, such as Rotation Forest.



	S1	S2	S3	S4
A1	0.024	5.124	7.682	0.715
A2	0.567	1.234	4.667	0.001
A3	4.567	6.231	12.435	4.456
A4	2.33	0.184	3.111	5.232

Intervals or Shapelets?

Property	Interval features	Shapelet distances
Where is the pattern?	At selected time positions	Best match anywhere in the series
Feature value	A statistic of the interval	Distance to a short pattern
Main assumption	Position is informative	Presence and shape are informative

Interval-based classifiers are designed to find smaller discriminatory regions that could be swamped by noise in other regions

Shapelet-based classifiers using sDist are designed to detect the presence of a shape anywhere in the series

Convolutional Features

- Define a short subseries/kernel

```
series = [1, 2, 4, 3, 1]
kernel = [-1, 0, 1]
```

- Slide along series and calculate the dot product to form an **activation map**

```
# First: -1*1 + 0*2 + 1*4 = 3
# Second: -1*2 + 0*4 + 1*3 = 1
# Third: -1*4 + 0*3 + 1*1 = -3
```

- Summarise/Pool the activation map into a small number of features

```
maximum = 3
Means 1/3
proportion_positive = 2 / 3
```

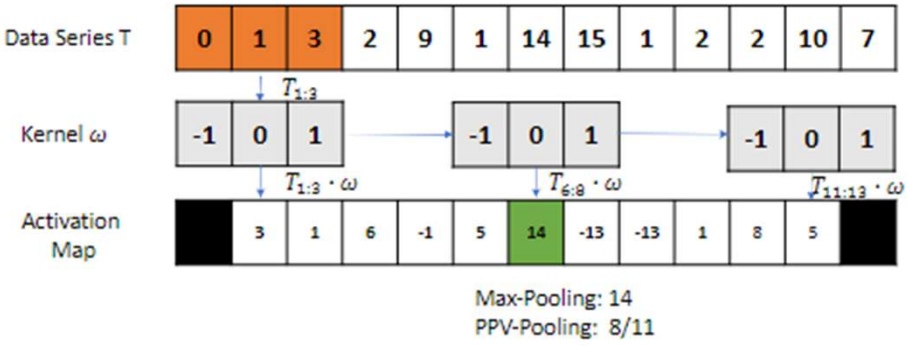
Fundamental component of Convolutional Neural Networks

Similar in concept to shapelets, but comes from Computer Vision/Image Classification (which uses 2D kernels)

Convolution/kernel based: ROCKET

A simple approach to TSC that does surprisingly well

1. Create a large number of random convolutions
2. Create feature vectors by pooling operations
3. Fit a linear classifier





Published: 13 July 2020

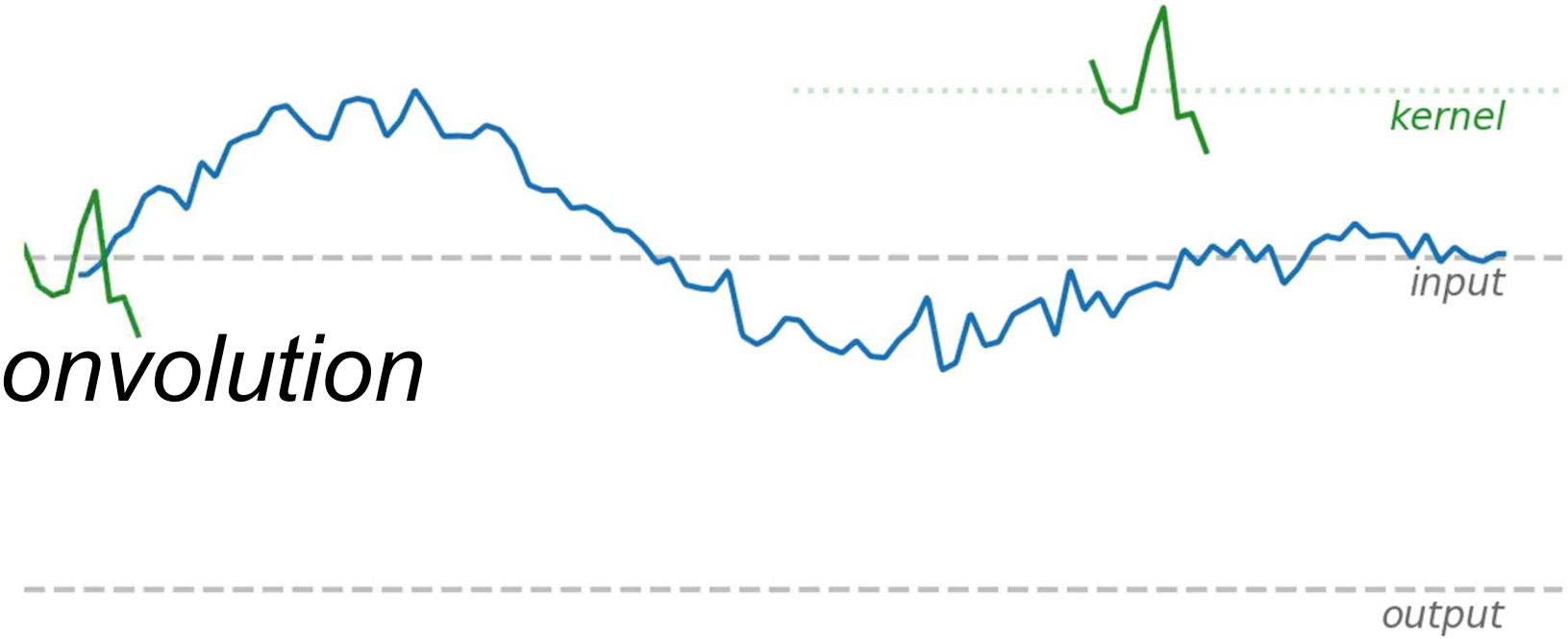
ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels

Angus Dempster, François Petitjean & Geoffrey I. Webb

MONASH University



Time Series



Convolution

Convolution-based Algorithms

Features are extracted from the kernel output series using **pooling operations** i.e. the ROCKET classifier uses:

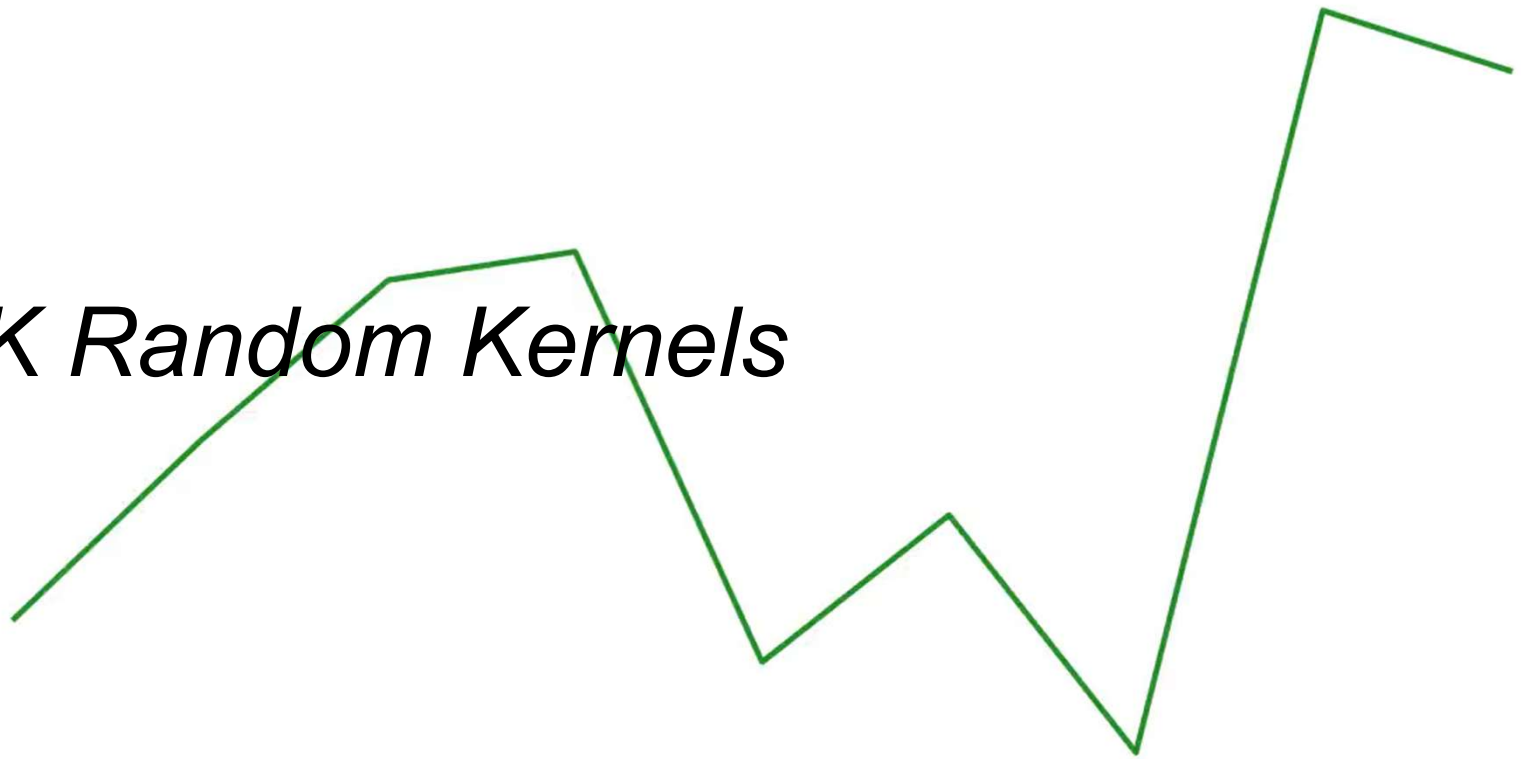
Max

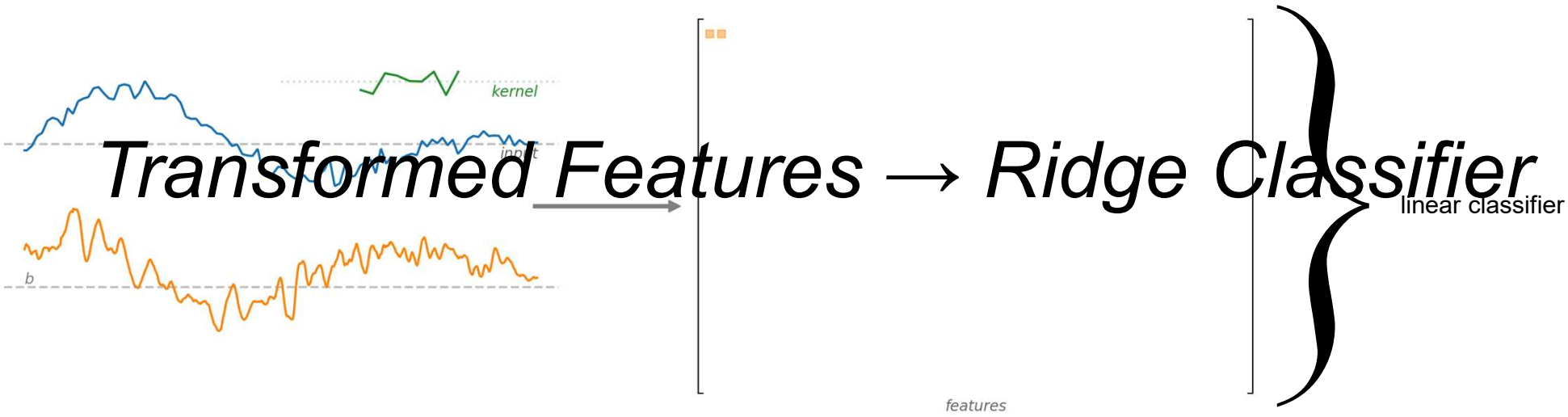


Proportion of Positive Values (PPV)



10K Random Kernels





Transformed Features → Ridge Classifier

ROCKET in aeon

```
from aeon.classification.convolution_based import (
    RocketClassifier,
)

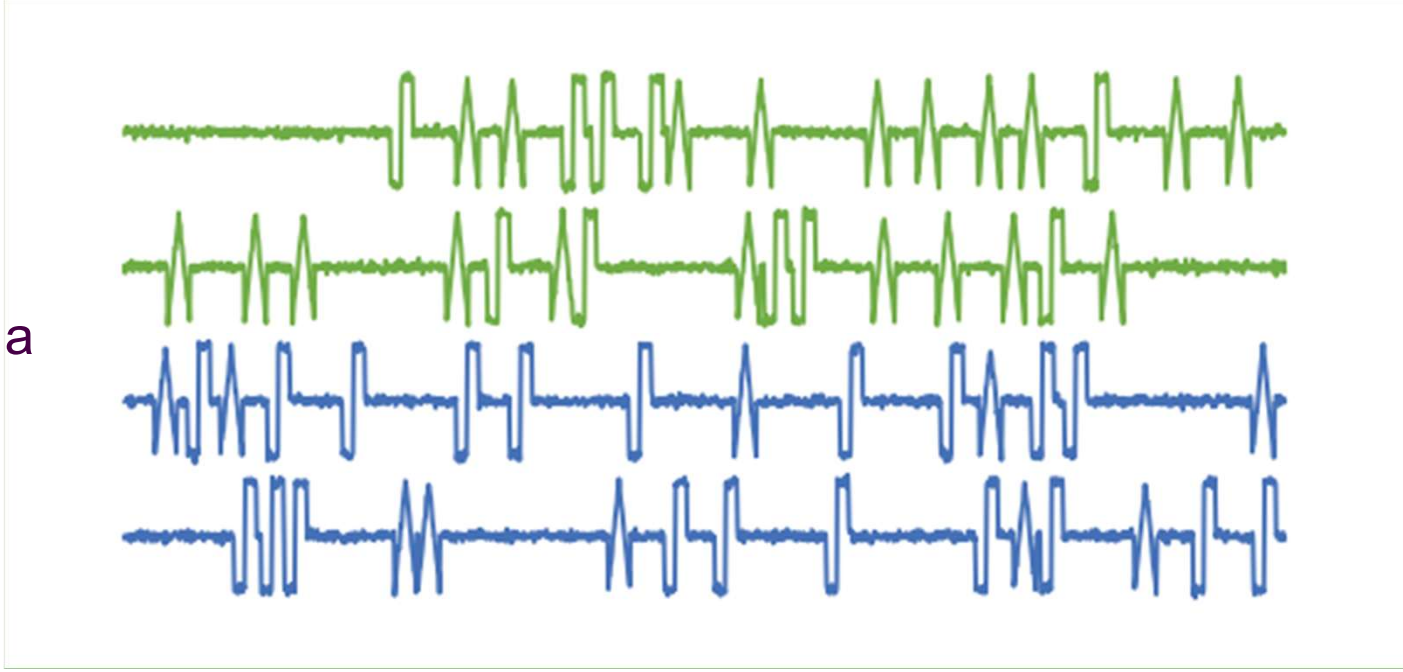
rocket = RocketClassifier(
    n_kernels=2000, random_state=0
)
rocket.fit(X_train, y_train)
pred = rocket.predict(X_test)
print(accuracy_score(y_test, pred))
```

Dictionary-based Time Series Classification

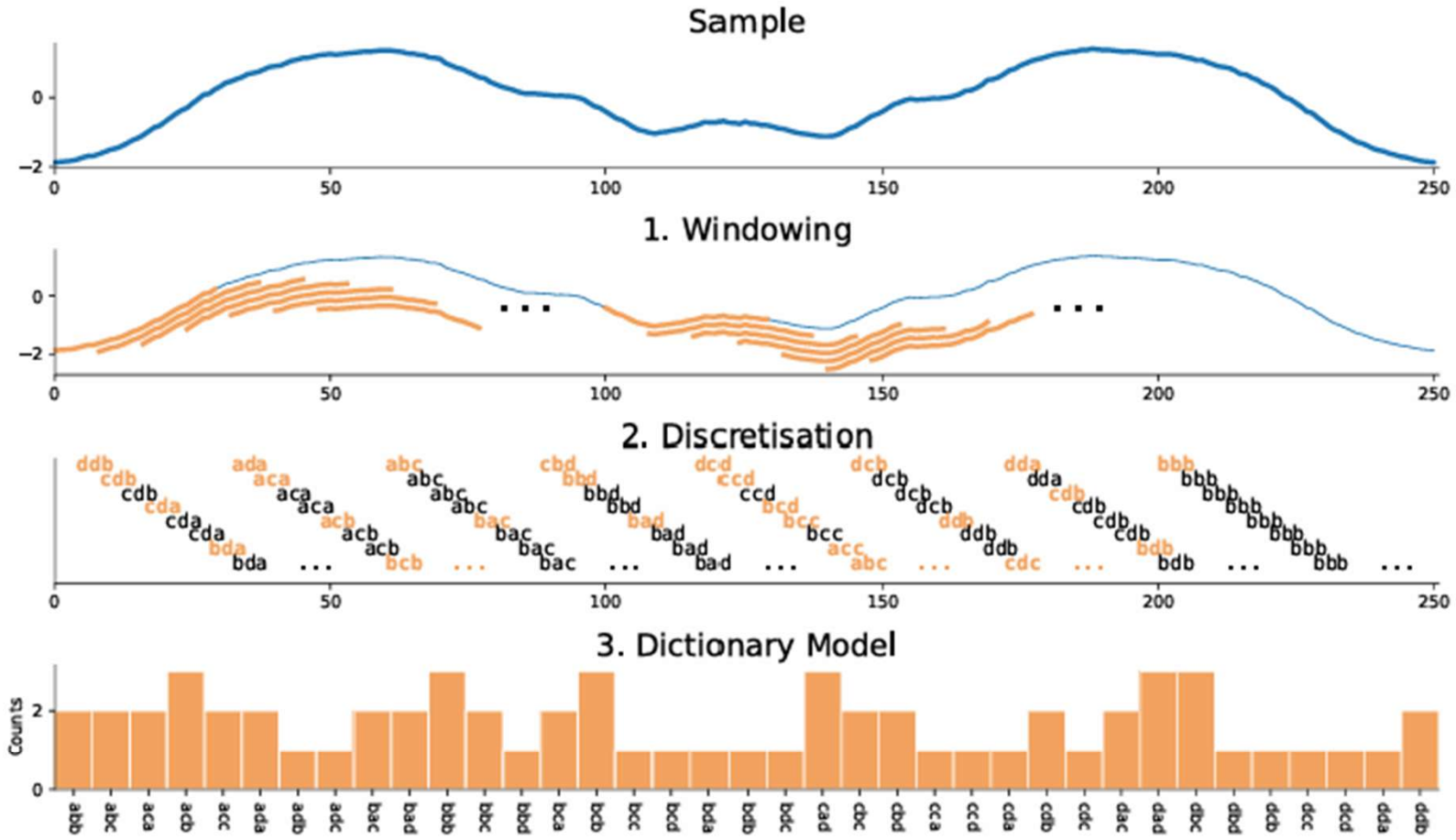
- Shapelets detect the presence or not of a pattern in series
- Convolutions detect presence (max) and proportion positive (PPV)

Class 1 Class 2

Dictionary-based explicitly look for repeated patterns through
1. Sliding a window across series
2. Conversion of pattern into a discrete word
3. Counting frequency of words



The BOSS



Schäfer, Patrick. "The BOSS is concerned with time series classification in the presence of noise." Data Mining and Knowledge Discovery 29 (2015): 1505-1530..

Dictionary-based TSC

- Each time series is transformed into a tabular dictionary (word, counts)
- Each word is a feature (column)
- This feature space can be high dimensional (depending on settings)
- It can be very sparse (many words don't occur)
- Most commonly used with a **nearest neighbour** classifier with a bespoke distance function
- This approach is directly derived from **bag-of-words** approaches used in **information retrieval**
- **Commonly called a state space model in IR**

Time Series Classification: whole series and interval based

		Examples
Distance-based	Compare whole series with a distance function, allow for some misalignment by using an elastic distance function	**1-NN with DTW** **ProximityForest**
Feature-based	Compare whole series using summary features	**Catch22** **FreshPRINCE**
Interval-based	Find summary features on different intervals	**TimeSeriesForest** **DrCIF**

Time Series Classification: Phase independent

		Examples
Shapelet-based	Find discriminatory shapes in the train data. Detect presence or not of the pattern	**ShapeletTransformClassifier** **RDST**
Convolution-based	Find convolutions/kernels and summarise activation to make features.	**ROCKET** **MRHydra**
Dictionary-based	Convert windows to discrete words, count occurrences, classify based on histograms	**BOSS** **WEASEL**